Nombre: Morales Gonzales Emmanuel – Boleta: 2025630602, Romero Sayún Jonathan - Boleta: 2025630298 ° Grupo: 3CV4

Ejercicio 3 — Investigación: qué es una base de datos

1. Dato, información y base de datos

1.1 Dato e información

Un dato es un hecho en bruto, registrable y carente de interpretación por sí mismo: la cadena 1976, el valor 38.5 o el carácter M. La información es el resultado de procesar esos datos dentro de un contexto que les asigna significado y los vuelve útiles para tomar una decisión: 38.5 °C de temperatura corporal registrados el 12 de marzo en el paciente con expediente 4471 ya no es un número, sino un indicio clínico. La distinción es operativa, no absoluta: la información de un nivel puede ser el dato de entrada del siguiente.

1.2 Definición formal de base de datos según dos autores

Elmasri y Navathe (2016) definen una base de datos como una colección de datos relacionados entre sí, donde por «datos» se entienden hechos conocidos susceptibles de ser registrados y que poseen un significado implícito. Los autores precisan que esta definición es demasiado amplia si no se le añaden tres propiedades implícitas: la base de datos representa algún aspecto del mundo real, al que denominan minimundo o universo del discurso; es una colección lógicamente coherente, con un significado inherente, y no un conjunto aleatorio de datos; y se diseña, construye y puebla con datos para un propósito específico, dirigida a un grupo de usuarios y a aplicaciones previamente identificadas.

Date (2004), en cambio, la define como una colección de datos persistentes utilizada por los sistemas de aplicación de una empresa determinada, y coloca el acento en dos elementos distintos: la persistencia, es decir, que los datos permanecen una vez que el programa que los creó ha terminado su ejecución, y la noción de empresa como cualquier organización — comercial, académica, hospitalaria o gubernamental— cuya operación requiere administrar información. Date insiste además en que el sistema de base de datos es, en esencia, un sistema computarizado de mantenimiento de registros.

Comparación. Ambas definiciones coinciden en lo esencial: un conjunto de datos organizado, con significado, gestionado de forma conjunta. Difieren en el énfasis. Elmasri y Navathe enfocan el problema desde la semántica y el diseño: lo que define a una base de datos es su correspondencia coherente con un fragmento acotado de la realidad y la intencionalidad de su construcción. Date lo enfoca desde la arquitectura de sistemas: lo que define a una base de datos es la persistencia y su papel como recurso compartido entre los sistemas de aplicación de una organización. La definición de Elmasri y Navathe es más útil para justificar la etapa de modelado conceptual; la de Date, para justificar la existencia del gestor como componente de

software independiente de las aplicaciones. Tomadas en conjunto, describen los dos ejes que sostienen la disciplina: el modelado del significado y la administración del almacenamiento.

2. Características de una base de datos

Integración. Los datos de las distintas áreas de la organización residen en un único depósito lógico, no en archivos dispersos por departamento. Esto permite que una consulta cruce información que antes estaba incomunicada, y es la condición previa de todas las demás características.

Persistencia. Los datos sobreviven a la ejecución de los programas que los manipulan y a la terminación de las sesiones de usuario; su ciclo de vida lo determina la organización, no el proceso (Date, 2004).

Redundancia controlada. El enfoque de bases de datos no elimina toda repetición: la controla. Cierta redundancia se conserva deliberadamente —claves foráneas, índices, vistas materializadas, desnormalización con fines de rendimiento — pero el sistema conoce esas copias y se hace responsable de mantenerlas consistentes, en lugar de dejar la tarea al programador (Elmasri & Navathe, 2016).

Integridad. Los datos deben satisfacer en todo momento las reglas del dominio que representan. Se expresa mediante restricciones declarativas: de dominio (el tipo y el rango admisibles), de entidad (toda tupla identificable de forma única, sin clave primaria nula), referencial (toda clave foránea apunta a una tupla existente) y de negocio o semánticas («la fecha de alta no puede ser posterior a la de baja»). La ventaja de declararlas en el esquema es que se aplican a todas las aplicaciones por igual.

Independencia de datos. Capacidad de modificar la definición de un nivel del esquema sin alterar el nivel superior. Se desarrolla en la sección 7.

Seguridad. Control de quién puede ver y modificar qué. Se articula mediante autenticación, autorización por privilegios sobre objetos (GRANT/REVOKE), vistas que restringen la proyección visible a cada rol, cifrado en reposo y en tránsito, y auditoría de accesos.

Concurrencia. Varios usuarios operan simultáneamente sobre los mismos datos sin corromper su consistencia. El gestor lo garantiza mediante el control de concurrencia —bloqueo en dos fases, marcas de tiempo o control multiversión (MVCC), que es el mecanismo de PostgreSQL— de modo que la ejecución entrelazada de varias transacciones resulte equivalente a alguna ejecución serial de las mismas.

Recuperación. Capacidad de restaurar un estado consistente tras una falla de transacción, del sistema o del medio de almacenamiento. Härder y Reuter (1983) formalizaron este dominio y acuñaron el acrónimo ACID (atomicidad, consistencia, aislamiento y durabilidad) como el

conjunto de garantías que una transacción debe ofrecer. Los mecanismos concretos son la bitácora de escritura anticipada (write-ahead log), los puntos de control (checkpoints) y las copias de seguridad.

3. Archivos frente a bases de datos

Antes del enfoque de bases de datos, cada aplicación definía y gestionaba sus propios archivos. El departamento de admisión de un hospital tenía su archivo de pacientes; el de facturación, el suyo; el de laboratorio, otro. Este esquema, llamado sistema de procesamiento de archivos, presenta tres patologías estructurales:

Redundancia. El mismo dato —el domicilio del paciente— se almacena en tantos archivos como aplicaciones lo necesiten. Esto desperdicia espacio, pero el costo real es el siguiente.

Inconsistencia. Cuando el paciente cambia de domicilio y sólo una de las tres aplicaciones lo actualiza, el sistema contiene dos verdades contradictorias sobre el mismo hecho, y ningún componente es responsable de detectarlo. La inconsistencia es la consecuencia inevitable de la redundancia no controlada.

Dependencia programa-datos. La estructura física del archivo —el orden de los campos, su longitud, el formato de codificación— está incrustada en el código de cada programa que lo lee. Agregar un campo obliga a recompilar y probar todos los programas que tocan ese archivo, aunque no usen el campo nuevo. El costo del cambio crece de forma no lineal con el número de aplicaciones. A esto se suman otras dificultades documentadas por Silberschatz et al. (2020): dificultad para acceder a los datos de formas no previstas al programar (cada consulta nueva exige un programa nuevo), aislamiento de los datos dispersos en formatos heterogéneos, problemas de atomicidad ante fallas a medio proceso, anomalías de acceso concurrente y ausencia de un mecanismo centralizado de seguridad.

Por qué surgieron los sistemas gestores. La respuesta a estas patologías fue extraer del código de las aplicaciones todo lo relativo a la definición, el almacenamiento y el acceso a los datos, y concentrarlo en una pieza de software independiente: el sistema gestor de bases de datos. La clave del cambio fue la separación entre la descripción de los datos y los programas que los usan: el esquema deja de vivir en el código y pasa a residir en el catálogo del propio gestor. De ahí se sigue todo lo demás —independencia de datos, consultas declarativas, control centralizado de integridad, seguridad y concurrencia—. El modelo relacional de Codd (1970) llevó esta separación a su forma más radical al desligar por completo la representación lógica de los datos de su organización física.

4. Usuarios de una base de datos

Usuario	*Responsabilidad principal*	*Permisos típicos*
Diseñador	Identificar los datos del minimundo, construir el modelo conceptual y lógico, definir entidades, atributos, relaciones y restricciones	Acceso al esquema en desarrollo; normalmente no opera sobre producción
Administrador (DBA)	Instalar y configurar el SGBD, otorgar y revocar accesos, afinar el rendimiento, planear respaldos y ejecutar recuperaciones, vigilar la integridad y la disponibilidad	Privilegios máximos: SUPERUSER, DDL, DCL, acceso al sistema operativo del servidor
Programador de aplicaciones	Desarrollar los programas que consultan y actualizan la base de datos a través de la API o del lenguaje anfitrión	DML (SELECT, INSERT, UPDATE, DELETE) sobre el subconjunto de objetos de su aplicación; rara vez DDL en producción
Usuario final	Consultar y capturar información para operar el negocio. Se subdividen en casuales, paramétricos (los que ejecutan transacciones predefinidas, como un cajero) y sofisticados (analistas que escriben sus propias consultas)	Acceso indirecto vía aplicación, o sólo lectura sobre vistas acotadas

A estos cuatro, Elmasri y Navathe (2016) añaden los llamados trabajadores entre bastidores: diseñadores e implementadores del propio SGBD, desarrolladores de herramientas y personal de operación y mantenimiento, que no usan la base de datos, sino que construyen el entorno que la sostiene.

El principio que ordena esta clasificación es el de mínimo privilegio: cada usuario debe recibir exclusivamente los permisos que su función exige. Que el programador de la aplicación de facturación pueda ejecutar DROP TABLE sobre el esquema clínico no es una comodidad, es un riesgo.

5. Ciclo de vida de una base de datos

Etapa	*Qué ocurre*	*Entregable*
1.Estudio de factibilidad y planeación	Se evalúa el alcance, los recursos, los plazos y la viabilidad técnica y económica	Plan del proyecto y estudio de factibilidad

2.Análisis de requerimientos	Entrevistas con los usuarios, revisión de documentación y formatos existentes, identificación de datos y de funciones	Documento de requerimientos de datos y de procesamiento
3.Diseño conceptual	Modelado independiente de cualquier gestor: entidades, atributos, relaciones y cardinalidades	Diagrama entidad-relación
4. Elección del SGBD	Comparación técnica, económica y operativa entre gestores candidatos	Justificación de la selección
5. Diseño lógico	Transformación del modelo conceptual al modelo de datos del gestor elegido y normalización	Esquema relacional normalizado
6. Diseño físico	Definición de estructuras de almacenamiento, índices, particionamiento y parámetros de ajuste	Esquema físico y script DDL
7. Implementación	Creación de la base de datos, carga inicial, desarrollo de las aplicaciones y de los mecanismos de seguridad	Base de datos operativa y aplicaciones
8. Pruebas y validación	Verificación de integridad, de rendimiento y de cumplimiento de los requerimientos	Informe de pruebas
9. Operación y mantenimiento	Monitoreo, respaldos, ajuste de rendimiento, corrección y evolución del esquema	Bitácoras, respaldos y documentación actualizada

El ciclo es iterativo, no lineal: los requerimientos descubiertos tardíamente obligan a revisar el modelo conceptual, y la etapa de operación alimenta de vuelta al diseño (Connolly & Begg, 2015).

6. Sistema gestor de bases de datos (SGBD)

Un SGBD es el conjunto de programas que permite definir, construir, manipular y compartir una base de datos entre diversos usuarios y aplicaciones (Elmasri & Navathe, 2016).

6.1 Módulos componentes

- **Procesador de consultas**, que incluye:
  - Analizador y traductor de DDL: interpreta las definiciones y las registra en el catálogo.
  - Compilador de DML: traduce las sentencias a un plan de evaluación.
  - Optimizador de consultas: selecciona, entre los planes equivalentes, el de menor costo estimado. El trabajo fundacional en este tema es el de Selinger et al. (1979).

    - Motor de evaluación: ejecuta el plan elegido.
- **Gestor de almacenamiento**, que incluye:
    - Gestor de autorizaciones e integridad: verifica permisos y restricciones.
    - Gestor de transacciones: garantiza las propiedades ACID.
    - Gestor de buffer: administra el traslado de páginas entre disco y memoria principal.
    - Gestor de archivos: organiza la asignación de espacio en disco.
- **Catálogo o diccionario de datos (metadatos):** almacena la definición del esquema, los usuarios, los privilegios y las estadísticas. Es lo que hace posible la independencia de datos, porque la descripción deja de estar en el código.
- **Índices y estructuras auxiliares**, y **archivos de bitácora** para la recuperación.

6.2 Lenguajes

Lenguaje	***Propósito***	***Sentencias típicas en SQL***
***DDL** (Definición de Datos)*	Definir y modificar la estructura de los objetos	CREATE, ALTER, DROP, TRUNCATE
***DML** (Manipulación de Datos)*	Consultar y modificar el contenido	SELECT, INSERT, UPDATE, DELETE
***DCL** (Control de Datos)*	Administrar privilegios y accesos	GRANT, REVOKE
***TCL** (Control de Transacciones)*	Delimitar las unidades atómicas de trabajo	BEGIN, COMMIT, ROLLBACK, SAVEPOINT

El DML admite dos estilos: procedimental, que especifica qué datos se necesitan y cómo obtenerlos, y declarativo o no procedimental, que especifica únicamente qué datos se necesitan y delega el cómo en el optimizador. SQL es declarativo, y en ello reside buena parte de su valor práctico.

6.3 Clasificación de los SGBD

- **Por modelo de datos**: relacionales (PostgreSQL, Oracle, MySQL), objeto-relacionales, jerárquicos, de red, y NoSQL en sus cuatro familias —clave-valor, documental, columnar y de grafos—.
- **Por número de usuarios**: monousuario y multiusuario.
- **Por número de sitios**: centralizados y distribuidos (estos últimos, homogéneos o heterogéneos).
- **Por costo y licencia**: libres y de código abierto frente a comerciales propietarios.
- **Por propósito**: de propósito general frente a especializados (series de tiempo, espaciales, vectoriales).
- **Por carga de trabajo**: OLTP, orientados a transacciones breves y numerosas, frente a OLAP, orientados a consultas analíticas sobre grandes volúmenes.

7. Sistema de base de datos: arquitectura y niveles

7.1 Arquitectura de tres niveles (ANSI/SPARC)

- **Nivel interno (físico).** Describe cómo se almacenan realmente los datos: organización de archivos, métodos de acceso, índices, compresión. Es el nivel más cercano al hardware.
- **Nivel conceptual (lógico).** Describe qué datos se almacenan y qué relaciones existen entre ellos, para la comunidad completa de usuarios, sin referencia al almacenamiento físico. Es el esquema de la base de datos propiamente dicho.
- **Nivel externo (de vistas).** Describe la porción de la base de datos que interesa a cada grupo de usuarios, y puede ocultar o reestructurar el resto. Hay tantos esquemas externos como perfiles de usuario se requieran.

El propósito de la separación en tres niveles es, precisamente, habilitar la independencia de datos.

7.2 Independencia de datos

Independencia física. Capacidad de modificar el esquema interno —añadir un índice, cambiar la organización del archivo, migrar a otro disco— sin alterar el esquema conceptual ni, por tanto, las aplicaciones. Es la más fácil de lograr.

Independencia lógica. Capacidad de modificar el esquema conceptual —añadir una columna o una entidad nueva — sin alterar los esquemas externos ni las aplicaciones existentes. Es más difícil de lograr, porque las aplicaciones dependen directamente de la estructura lógica; las vistas son el mecanismo que la hace posible.

El precio de la independencia es la transformación de esquemas (mapping): cada consulta debe traducirse del nivel externo al conceptual y de éste al interno, lo que consume tiempo de procesamiento.

7.3 Arquitectura centralizada y cliente-servidor

En la arquitectura centralizada, los datos, el SGBD y las aplicaciones residen en una única computadora, a la que los usuarios acceden mediante terminales sin capacidad de cómputo. Es simple de administrar y asegurar, pero constituye un punto único de falla y no escala.

En la arquitectura cliente-servidor, el servidor aloja la base de datos y el gestor, y atiende peticiones de múltiples clientes que se comunican por red. Admite dos variantes:

- **De dos capas**. El cliente contiene la interfaz y la lógica de negocio, y se conecta directamente al servidor de base de datos mediante una interfaz como ODBC o JDBC. Es sencilla, pero distribuir un cambio de lógica exige actualizar todos los clientes.
- **De tres capas.** Se interpone un servidor de aplicaciones entre el cliente y el servidor de base de datos. El cliente sólo presenta; la lógica de negocio y las credenciales de acceso quedan del lado del servidor. Mejora la escalabilidad y la seguridad, y es el esquema

dominante en las aplicaciones web actuales. Su extensión natural son las arquitecturas de n capas y las distribuidas en la nube.

El entorno construido en el Ejercicio 2 es un ejemplo de arquitectura cliente-servidor: el contenedor pg-practica1 actúa como servidor, y psql, DBeaver o pgAdmin como clientes que se conectan a través del puerto 5432 publicado.

8. Modelos de datos

Un modelo de datos es un conjunto de conceptos que permiten describir la estructura de una base de datos —los tipos de datos, las relaciones y las restricciones— junto con un conjunto de operaciones básicas para especificar consultas y actualizaciones (Elmasri & Navathe, 2016).

8.1 Importancia

Su valor es doble. Primero, provee un lenguaje común entre el analista, el cliente y el programador: permite discutir la estructura de la información sin discutir todavía su implementación. Segundo, es el mecanismo que hace posible la abstracción: al describir los datos en términos de conceptos y no de bytes, separa el significado de la representación, que es la condición de la independencia de datos. Un modelo bien construido anticipa errores cuya corrección posterior, con la base de datos ya poblada, resulta costosísima.

8.2 Clasificación por nivel de abstracción

- **Modelos conceptuales o de alto nivel.** Describen los datos como los percibe el usuario, mediante entidades, atributos y relaciones. Son independientes del gestor. El caso paradigmático es el modelo entidad-relación de Chen (1976) y su extensión, el modelo entidad-relación extendido (EER), que añade especialización, generalización y categorías.
- **Modelos lógicos o de representación.** Describen los datos con estructuras que el gestor implementa directamente, pero sin descender al almacenamiento: el modelo relacional de Codd (1970) —tablas, tuplas, atributos, claves—, y los históricos modelos jerárquico y de red.
- **Modelos físicos o de bajo nivel.** Describen cómo se almacenan los datos en el medio: formatos de registro, orden, rutas de acceso, índices B+.

8.3 Clasificación por estructura

Relacional. Los datos se organizan en relaciones (tablas) y los vínculos se expresan mediante valores de clave foránea, no mediante apuntadores. Dominante desde los años ochenta.

Jerárquico. Los datos se organizan en una estructura de árbol con relaciones padre-hijo 1:N. Eficiente en su nicho, pero incapaz de representar de forma natural las relaciones M:N.

De red. Generalización del jerárquico mediante conjuntos y apuntadores, que sí admite relaciones M:N, a costa de una navegación compleja y de una fuerte dependencia física.

Orientado a objetos y objeto-relacional. Incorporan identidad de objeto, tipos complejos, herencia y encapsulamiento. PostgreSQL pertenece a esta última categoría.

Semiestructurados y NoSQL. Documentales (JSON, XML), clave-valor, columnares y de grafos. Relajan el esquema rígido y, con frecuencia, las garantías ACID, a cambio de escalabilidad horizontal y flexibilidad. Su estudio comparativo es abundante en la literatura reciente (Davoudian et al., 2018; Lu & Holubová, 2019).

Referencias

Chen, P. P. (1976). The entity-relationship model: Toward a unified view of data. ACM Transactions on Database Systems, 1(1), 9–36. https://doi.org/10.1145/320434.320440

Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377– 387. https://doi.org/10.1145/362384.362685

Connolly, T. M., & Begg, C. E. (2015). Database systems: A practical approach to design, implementation, and management (6.ª ed.). Pearson Education.

Date, C. J. (2004). An introduction to database systems (8.ª ed.). Addison-Wesley.

Davoudian, A., Chen, L., & Liu, M. (2018). A survey on NoSQL stores. ACM Computing Surveys, 51(2), Artículo 40, 1–43. https://doi.org/10.1145/3158661

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of database systems (7.ª ed.). Pearson Education.

Härder, T., & Reuter, A. (1983). Principles of transaction-oriented database recovery. ACM Computing Surveys, 15(4), 287–317. https://doi.org/10.1145/289.291

Lu, J., & Holubová, I. (2019). Multi-model databases: A new journey to handle the variety of data. ACM Computing Surveys, 52(3), Artículo 55, 1–38. https://doi.org/10.1145/3323214

Selinger, P. G., Astrahan, M. M., Chamberlin, D. D., Lorie, R. A., & Price, T. G. (1979). Access path selection in a relational database management system. Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data, 23–34. https://doi.org/10.1145/582095.582099

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database system concepts (7.ª ed.). McGraw-Hill Education.